

Detailed Research Plan

Programme: MSc in Computer Science

Track: Computer Science (Software Architecture)

Student: Hamed Mohamed (DG1EWF)

Supervisor: Morse Gregory Reynolds

Working title: *Migrating Spring Boot MVC Systems to Java 21 Virtual Threads: Architecture, Risks, and Engineering Guidelines*

The approved topic registration identifies this MSc Software Architecture track, the title above, and the synopsis describing a Spring Boot MVC demonstration application, k6/JFR benchmarking, platform/naive/refactored Virtual-Thread comparisons, and a migration checklist. No scope changes have been made since that registration.

1. Research aim and motivation

Java 21 Virtual Threads make it possible to preserve a conventional thread-per-request programming model while supporting substantially more concurrent blocking tasks than a fixed platform-thread pool. This makes them attractive for existing Spring Boot MVC applications whose workloads contain database calls, remote-service calls, and other blocking operations. Virtual Threads do not, however, remove synchronization, resource-pool limits, or architectural coupling. An application can therefore become slower or less predictable after migration if legacy code prevents Virtual Threads from unmounting or if downstream resources become saturated.

The aim of this thesis is to produce an evidence-based migration assessment for a representative Spring Boot MVC application. The study will identify which application-level patterns create measurable risk, quantify their effect under controlled workloads, and evaluate whether targeted refactoring restores predictable performance. The practical output will be a migration checklist grounded in the experiment rather than a claim that enabling `spring.threads.virtual.enabled` is universally beneficial.

The primary research focus is **thread pinning caused by intrinsic synchronization around blocking work**. `ThreadLocal` usage and blocking JDBC access are treated as separate architectural risks: they may affect memory, context propagation, or resource contention, but they will not be labelled pinning causes unless JFR evidence supports that conclusion. This distinction is necessary for technically valid conclusions.

2. Exact research questions and expected answers

RQ1

How does an intrinsic synchronization mechanism surrounding blocking work affect the throughput, latency, error rate, and Virtual Thread pinning behaviour of a Spring Boot MVC application compared with platform threads and a Virtual-Thread-compatible lock?

The main manipulated factor is the payment-service synchronization mechanism:

1. platform-thread execution with the legacy synchronized implementation;
2. Virtual-Thread execution with the same synchronized implementation;
3. Virtual-Thread execution after replacing intrinsic synchronization with `ReentrantLock`.

The expected answer is conditional rather than absolute: intrinsic synchronization should increase `jdk.VirtualThreadPinned` events and degrade performance under sufficient concurrency, while the refactored implementation should reduce application-level pinning. The magnitude of the effect must be reported with repeated measurements and uncertainty.

RQ2

Which architectural patterns in a legacy Spring Boot MVC data path must be audited or refactored before enabling Virtual Threads, and what measurable risks remain after the primary

pinning defect is removed?

The study will examine three deliberately contrasting patterns:

- intrinsic synchronization around a blocking payment call;
- request-context state stored in a manually managed `ThreadLocal`;
- blocking JDBC access and fixed database-resource capacity on the orders path.

The answer will distinguish three mechanisms: carrier-thread pinning, request-context/resource-lifecycle risk, and downstream connection or queue contention. A pattern will be included in the final migration guidance only when its mechanism, observable symptom, and mitigation are supported by code inspection and experiment evidence.

Practical contribution

The thesis will derive a migration procedure from the two research questions: establish a platform baseline, enable Virtual Threads without changing application code, locate pinning with JFR, audit context and blocking-resource usage, refactor the confirmed defect, and repeat the measurements. This checklist is a contribution for practitioners, but it is not treated as a third research question.

Literature-grounded gap

JEP 444 and the Java 21 documentation establish the intended Virtual Thread execution model and its pinning limitations, while Spring Boot documentation explains how framework configuration enables Virtual Threads without rewriting application code. Tomcat and HikariCP documentation establish the independent server-thread and database-pool constraints that can confound a comparison. The bibliography review also places this problem in the longer history of thread-based and event-driven server design. The unresolved gap addressed here is narrower and empirical: a reproducible, mechanism-focused study of how a legacy Spring MVC-style application behaves when a blocking synchronized path is migrated naively, then refactored, while a clean endpoint and a data/context path provide controls. The study therefore connects runtime observability (`jdk.VirtualThreadPinned`) to application-level migration decisions rather than treating throughput change as sufficient evidence.

3. Research mode, artifact, and scope

This is an **empirical engineering/system study**. The primary artifact is a versioned Spring Boot application with intentionally controlled workload paths. The repository contains the source, Maven build, application configuration, k6 workload, benchmark orchestration script, JFR recordings, raw k6 JSON outputs, and analysis documents.

The application provides three endpoint roles:

Endpoint	Experimental role	Main phenomenon
GET /orders	Slow data path	300 ms simulated blocking work, JDBC access, and cleaned-up <code>ThreadLocal</code> context
POST /payments	Pinning treatment	500 ms simulated external call protected first by intrinsic synchronization and then by <code>ReentrantLock</code>
GET /products	Clean control	Fast JDBC query without the intentionally introduced application anti-patterns

The local implementation uses H2 and simulated delays. Docker/PostgreSQL configuration is retained as deployment infrastructure, but PostgreSQL-specific conclusions will not be claimed unless the PostgreSQL

configuration is actually executed and its results are separately recorded. The study does not claim production-scale behaviour, universal superiority of Virtual Threads, or that replacing every `synchronized` construct with `ReentrantLock` is always semantically correct.

4. Experimental design and procedure

Independent variables

- **Execution mode:** platform threads, naive Virtual Threads, or refactored Virtual Threads.
- **Synchronization implementation:** intrinsic synchronization versus `ReentrantLock` on the payment path.
- **Load level:** 50, 100, and 200 k6 virtual users.
- **Endpoint role:** orders, payments, and products.

The platform baseline will use Virtual Threads disabled and a documented Tomcat platform-thread configuration. The naive Virtual-Thread condition will preserve the legacy synchronized implementation. The refactored condition will change only the synchronization implementation initially, preserving the workload and endpoint contracts. Any later pool-size experiment will be labelled an extension or sensitivity analysis rather than silently mixed with the main comparison.

Workload

The k6 script ramps to the selected user level for 15 seconds, holds the target for 30 seconds, and ramps down for 10 seconds. Each virtual user invokes `/orders`, `/payments`, and `/products` in the same iteration, then pauses briefly. The primary comparison will use the steady-state hold interval. The script records request duration, HTTP failures, endpoint checks, and custom endpoint trends.

To improve validity before final thesis measurements, the workload will be revised or post-processed so that endpoint-specific results are not confused with the aggregate request stream. Where possible, each endpoint will also be benchmarked in an isolated run. This will test whether an apparent control-endpoint degradation is genuine cross-endpoint interference rather than an artefact of a mixed workload.

Observability

Each run will record:

- Java version, Spring Boot version, operating system, CPU and memory;
- Git commit, configuration mode, endpoint, VU level, and run duration;
- throughput, p50, p95, p99 or a clearly labelled maximum, and error rate;
- JFR recording and the exact `jfr print` command used;
- count, duration, and stack-trace classification of `jdk.VirtualThreadPinned` events;
- application logs and k6 raw JSON output.

The current repository contains JFR files and k6 JSON files from earlier runs. Before final analysis, the selected files will be mapped to commits and configurations, and all headline event counts will be regenerated from the recordings. Narrative values such as approximately 485 or 12 pinning events are provisional until this mapping is complete.

5. Analysis and decision rules

For each endpoint and load level, the analysis will compare the three execution modes using absolute values and relative change from the platform baseline. At least three valid repetitions per condition will be targeted where time and machine stability permit. If only historical single runs are available, they will be labelled exploratory and will not be presented as estimates of stable population performance.

The primary outcomes are:

1. throughput in requests per second;

2. p50, p95, and p99 latency, with timeout-censored observations reported separately;
3. HTTP error rate;
4. application-level `jdk.VirtualThreadPinned` count and total pinned duration.

The primary decision rule for RQ1 is that the synchronized Virtual-Thread condition demonstrates a pinning-related degradation only if: (a) JFR identifies application stack traces at the intrinsic lock, (b) the condition differs from the refactored condition in pinning measurements, and (c) the performance difference is reproducible under the same workload and environment. A correlation alone will not be treated as proof of causation.

For planning purposes, a change will be called **material** when it is at least a 20% throughput difference, a two-fold p95 latency difference, or a 5-percentage-point error-rate difference at the same load level. These thresholds are interpretation thresholds, not promises that the experiment will produce a positive result. A result below the threshold will be reported as practically small even if its direction is consistent. Pinning evidence remains a separate criterion and requires stack-trace attribution.

For RQ2, each proposed anti-pattern will receive an evidence classification:

- **confirmed pinning source:** application stack trace in `jdk.VirtualThreadPinned` with corresponding performance effect;
- **resource/context risk:** mechanism supported by code and resource observations, but not labelled pinning without JFR evidence;
- **not demonstrated:** plausible in general but not established by this experiment.

For repeated runs, the analysis will report the median, interquartile range, and bootstrap 95% confidence interval for each primary metric. Run-level differences will be retained rather than pooling all requests across runs. If fewer than three valid repetitions are available, the result will be descriptive and exploratory, not a stable estimate. Large latency changes will be accompanied by timeout/error counts and workload completion counts, since a lower latency among only the requests that completed successfully can be misleading. No “improvement” claim will be based on throughput alone when error rates differ materially.

6. Validity threats and mitigations

Internal validity. The endpoints share one application process and one workload script, so cross-endpoint interference and warm-up effects may confound results. The mitigation is endpoint-isolated runs, a fixed warm-up/hold protocol, randomized or balanced condition order where practical, and recording the exact commit and configuration.

Measurement validity. JFR event thresholds and k6 percentile definitions affect conclusions. The project will fix the JFR settings, report the threshold, use `jfr print` output as the event source, and distinguish p99 from maximum latency. The benchmark script and raw outputs will remain part of the artifact.

Confounding by database capacity. H2, JDBC connection limits, and artificial sleeps may dominate some results. The thesis will state clearly which observations concern synchronization and which concern downstream resource contention. A small sensitivity experiment varying the connection-pool limit may be added if it can be executed consistently; it will not be presented as completed evidence otherwise.

External validity. One application and one development machine cannot represent all Spring Boot systems. The contribution will therefore be framed as a mechanism-focused case study and migration method, with transfer limits explicitly stated. The literature review and official JDK/Spring sources will provide the broader context.

Construct validity of ThreadLocal claims. The demo cleans up its `ThreadLocal` in a `finally` block. Therefore the study can demonstrate the presence of a request-context pattern and discuss its scalability/lifecycle implications, but it cannot claim a measured leak or pinning effect without a separate memory experiment.

Reproducibility. Historical result documents contain placeholders and some claims that require reconciliation with raw artifacts. The final thesis dataset will include a run manifest, source commit, configuration, machine

information, raw k6 JSON, JFR file, extraction output, and analysis table for every retained result.

7. Work plan, risks, and fallback

Period	Activity	Inspectable output	Dependency/fallback
Week 1	Evidence freeze and correction	Run manifest, commit/configuration mapping, corrected claims list	If historical runs cannot be mapped, mark them exploratory
Week 2	Method hardening	Endpoint-isolated workload, metadata template, fixed JFR extraction procedure	Retain mixed-workload runs only as secondary evidence
Weeks 3–4	Baseline and naive-condition reproduction	Validated platform and naive Virtual-Thread runs	Narrow claims to available valid conditions
Week 5	Refactoring comparison	Refactored runs and stack-trace classification	Report incomplete repetitions as a limitation
Week 6	Sensitivity analysis and synthesis	Optional pool-capacity analysis, RQ answers, migration checklist	Omit the extension if it is not reproducible
Week 7 onward	Thesis drafting and review	Methodology chapter, results tables, reproducibility package	Freeze scope before final writing

The main risks are unavailable historical environments, unstable local performance, insufficient repeated runs, and disagreement between narrative documents and raw recordings. The fallback is to narrow claims to a reproducible single-machine case study, retain only traceable measurements, and report unresolved items as limitations rather than inventing precision. No production data, human participants, or personally identifiable information are required.

The project uses public open-source software and documentation, and the experiment generates synthetic H2 records rather than collecting personal or confidential data. Any PostgreSQL image, library, or dataset used in an extension will be recorded with its version and licence. No human-subject approval or participant consent is required for the planned system measurements.

8. Integrity and AI-use transparency

The student remains responsible for the research questions, experimental design, code, interpretation, and final submitted text. AI assistance may be used for drafting, code navigation, and editorial feedback, but every technical claim and citation will be checked against the repository or an authoritative source. Point-of-use disclosure and the retained interaction log will follow the current ELTE AI policy and the supervisor’s instructions. AI-generated suggestions will not be treated as experimental evidence, and no claim of a run, result, or supervisor approval will be made without a retained artifact or direct confirmation.

For the final thesis package, the supplied ELTE declaration form requires selecting either “no AI tools used” or “AI tools used” and, in the latter case, recording the purpose, how the tool was used, tool name, and thesis location. It also requires the student to confirm responsibility for the content, verification and proper citation of AI-assisted material, and compliance with the applicable academic regulations and Dean’s instruction. This plan therefore treats the interaction log and final declaration as submission artifacts, not as evidence of experimental results.

9. Expected contribution

The expected result is not simply that Virtual Threads are faster. The thesis will provide:

1. a controlled Spring Boot MVC artifact containing representative migration hazards;
2. a traceable comparison of platform, naive Virtual-Thread, and refactored Virtual-Thread execution;
3. a JFR-based procedure for locating application-level pinning;
4. a defensible separation between pinning, context-management, and database-resource risks;
5. a practical migration checklist whose recommendations are tied to measurable evidence and explicit limitations.

The plan is designed to become the thesis methodology chapter without changing its scientific logic. Any scope expansion, including connection-pool sensitivity experiments or PostgreSQL deployment results, will be recorded as an explicit change rather than merged retrospectively into the original claims.